

AWS vs GCP

大数据与分析产品全景对比

10 大类别 · 逐产品优缺点 · 湖仓 / 表格式 (含 S3 Tables)

基于 AWS / GCP 官方文档 · 2026-08 · 数据随产品更新，以官方为准

总览：两种架构哲学

AWS 多而全 (purpose-built) vs GCP 少而精 (serverless 为主)

维度	AWS	GCP
架构哲学	每种负载一个专门服务，组合灵活、生态广	BigQuery 一个平台包揽多数分析，免运维
计费主基调	按实例 / 节点时长 或 serverless 按用量	serverless 为主 (扫描量 /slot) ，无需管集群
独有强项	搜索 (OpenSearch)、托管 Kafka(MSK)、精细选型	数仓 (BigQuery)、批流一体 (Dataflow/Beam)、语义层 (Looker)
开放格式	S3 Tables 原生 Iceberg、引擎中立	Iceberg/Delta/Hudi 内建，但引擎偏 BigQuery

⚠ 4 个近期改名 (已核实官方文档)

做技术选型 / 沟通时用新名，避免踩坑

Dataproc → Google Cloud Managed Service for Apache Spark

官方文档已重定向，旧名仍在用

QuickSight → Amazon Quick Sight

隶属新品牌 Amazon Quick(AI 套件)，原 Q 自然语言并入

Cloud Composer → Managed Airflow (Gen3)

文档新称谓

BigLake Iceberg tables → Apache Iceberg managed tables

BigQuery 官方新称谓

1. 数据仓库

Amazon Redshift vs Google BigQuery

AWS • Amazon Redshift

定位：PB 级托管数仓，Provisioned+Serverless 两形态

✓ 优点

- Serverless 无需管集群，按需 / 预留灵活
- 与 S3/Glue/SageMaker 深度集成
- RMS 存储按 GB/ 月独立计费，成本可控

✗ 缺点

- Provisioned 仍需选节点 / 管容量
- Python UDF 2026-06-30 后停止支持

GCP • Google BigQuery

定位：全托管 Serverless、存算分离、AI-ready 数据平台

✓ 优点

- 真 Serverless、存算彻底分离、资源动态互不干扰
- 计费双模型可切 (按 TiB 扫描 / 按 slot)
- 原生 AI/ML、搜索、开放表格式内建

✗ 缺点

- on-demand 按扫描字节，未裁剪大表易失控
- 深度绑定 GCP 生态

选型建议：重 AWS 生态 / 需精细节点控制 → Redshift；追求零运维 + 内建 AI+ 弹性计费 → BigQuery

2. 交互式查询

Amazon Athena vs BigQuery 外部表 / BigLake

AWS · Amazon Athena

定位：Serverless SQL 直查 S3，另含 Serverless Spark

✓ 优点

- 完全 Serverless、按查询付费、无基础设施
- 自动并行扩展，大数据集秒级返回
- 内建 Serverless Spark notebook，SQL+Spark 双模

✗ 缺点

- 按扫描量计费，未用列存 / 分区易贵
- 高并发 / 复杂长任务性能受限（待确认配额）

GCP · BigQuery 外部表 / BigLake

定位：BigQuery 外部表 / BigLake 查 GCS 及开放格式并统一治理

✓ 优点

- 一份数据多引擎共享，支持 Iceberg/Delta/Hudi
- 与 BigQuery 治理（列级安全 / 脱敏 / 血缘）统一
- 可与原生表混查，体验一致

✗ 缺点

- 外部表相比原生表有性能 / 功能限制
- 概念较多（外部表 / BigLake / Iceberg）门槛略高

选型建议：纯 S3 即席查询 / 轻量按需 → Athena；已用 BQ 且要跨引擎共享开放格式 → BigLake / 外部表

3. 托管 Spark/Hadoop

Amazon EMR vs Managed Service for Apache Spark

AWS • Amazon EMR

定位：托管 Spark/Hive 等，EMR on EC2/EKS + EMR Serverless

✓ 优点

- EMR Serverless 自动配资源、跑完即释放
- EC2/EKS/Serverless 多部署形态灵活
- 可预初始化资源获秒级交互响应

✗ 缺点

- EMR on EC2 仍需选实例 / 调优集群
- 版本 / 框架组合与调参复杂度高

GCP • Managed Service for Apache Spark

定位：原 Dataproc，托管 Spark，1 分钟起停、秒级计费

✓ 优点

- 按 vCPU \$0.01/ 小时、秒级计费、1 分钟最小计费
- 集群快速创建、用完关机省钱
- 支持抢占式实例进一步降本

✗ 缺点

- 传统集群形态仍需管理生命周期
- 生态偏 Spark，Hadoop 组件广度不如 EMR

选型建议：需多框架 / 多部署 + 深 AWS 集成 → EMR；主跑 Spark 要极简计费快速起停 → Managed Spark

4. 流式处理引擎

Kinesis + Managed Flink vs Dataflow (Apache Beam)

AWS • Kinesis + Managed Flink

定位：Kinesis 摄取 + Managed Flink 做流处理（多语言）

✓ 优点

- Managed Flink 托管底层（弹性 / AZ 容错 / 快照）
- 基于开源 Flink，可用全部算子 / 库
- 与 Kinesis/MSK/S3 原生打通

✗ 缺点

- 需自行组合 Streams+Flink 两个服务
- Flink 应用调优 / 状态管理有学习曲线

GCP • Dataflow (Apache Beam)

定位：统一批流处理，同一 Beam 模型，默认 exactly-once

✓ 优点

- 批流统一同一模型，一套代码两用
- 全托管、自动分配 / 回收 worker，用完即删
- 默认 exactly-once，可选 at-least-once 降本

✗ 缺点

- 绑定 Beam 模型，跨云迁移成本高
- 复杂 pipeline 资源 / 成本调优需经验

选型建议：已用 Flink/AWS 流栈 → Kinesis+Managed Flink；要批流一体极简运维 → Dataflow

5. 消息 / 事件摄取

Kinesis / Amazon MSK vs Pub/Sub

AWS • Kinesis / Amazon MSK

定位：Kinesis(原生流摄取) / MSK(托管 Kafka)

✓ 优点

- MSK 提供托管标准 Kafka(生态兼容可迁移)
- Kinesis 与 Firehose/Flink/Lambda 无缝集成
- 覆盖 Kafka 兼容与 AWS 原生两类需求

✗ 缺点

- Kinesis 分片需容量规划与扩缩管理
- MSK 需一定 Kafka 运维知识

GCP • Pub/Sub

定位：异步可扩展消息服务，解耦生产消费，延迟约 100ms

✓ 优点

- 全托管、自动扩展、无分片 / 容量规划负担
- 异步 pub/sub 提升系统灵活与健壮
- 天然对接 Dataflow/BigQuery 做流分析

✗ 缺点

- 非 Kafka 协议，已有 Kafka 应用迁移需改造
- 端到端约 100ms，超低延迟场景需评估

选型建议：需 Kafka 兼容→ MSK；AWS 原生流→ Kinesis；要零运维全托管事件总线→ Pub/Sub

6. ETL/ 数据集成

AWS Glue vs Cloud Data Fusion

AWS • AWS Glue

定位：Serverless 数据集成 (爬取 / 编目 /Spark ETL)

✓ 优点

- Serverless 、无需管理基础设施
- 内建 Data Catalog 统一元数据 (供 Athena/Redshift 共享)
- 与 AWS 分析栈深度打通

✗ 缺点

- 可视化编排弱于图形化工具 (偏代码)
- Spark 作业冷启动与调优需经验

GCP • Cloud Data Fusion

定位：基于开源 CDAP 的可视化图形化 ETL/ELT ，丰富插件

✓ 优点

- 图形化拖拽建管道、大量预置插件、低代码
- 基于开源 CDAP ，可移植性好
- 支持批 / 流与多源连接 (SAP/Salesforce 等)

✗ 缺点

- 底层跑在 Dataproc ，实例常驻产生持续费用
- 相对重，启动 / 实例管理有开销

选型建议：代码化 Serverless ETL+AWS 编目→ Glue ； 低代码可视化 + 多源连接→ Data Fusion

7. workflow编排

Amazon MWAA vs Managed Airflow (Composer Gen3)

AWS • Amazon MWAA

定位：托管 Apache Airflow ， Python 建管道、自动伸缩

✓ 优点

- 全托管 Airflow 、自动伸缩、无基础设施负担
- 建环境时选 Airflow 版本、自动装配
- 集成 AWS 安全服务快速安全访问数据

✗ 缺点

- 环境按容量常驻计费，空闲也有成本
- Airflow 版本升级 / 兼容需规划

GCP • Managed Airflow (Composer Gen3)

定位：全托管 Airflow(Gen3) ，可跨云及本地编排

✓ 优点

- 全托管、原生 Airflow Web UI 与 CLI
- 可跨云 / 本地编排 pipeline
- Gen3 持续迭代新特性

✗ 缺点

- 底层环境常驻、成本随规模上升
- 版本代际 (Gen1/2/3) 迁移需注意兼容

选型建议：功能相当，按所在云选：AWS→MWAA ， GCP→Composer/Managed Airflow

8. BI/ 可视化

Amazon Quick Sight vs Looker / Looker Studio

AWS • Amazon Quick Sight

定位：交互式 BI(现属 Amazon Quick AI 套件) ，可嵌入分析

✓ 优点

- Serverless 、按会话 / 容量计费，可嵌入应用
- 归入 Amazon Quick 后与 AI 代理 / 自然语言打通
- 与 AWS 数据源原生连接

✗ 缺点

- 品牌重组仍在演进，功能边界待稳定
- 建模 / 语义层能力弱于 Looker

GCP • Looker / Looker Studio

定位：Looker(LookML 语义层企业 BI) / Looker Studio(轻量免费)

✓ 优点

- LookML 语义层统一指标定义、单一真相源
- 强嵌入分析与 API 、可作数据应用平台
- Looker Studio 免费轻量上手快

✗ 缺点

- Looker 企业版授权成本高、LookML 有学习曲线
- Looker 与 Looker Studio 两套产品易混淆

选型建议：要治理化语义层 / 企业建模→ Looker ； 轻量免费→ Looker Studio ； AWS 生态 +AI 化 BI→Quick Sight

9. 搜索 / 日志分析

Amazon OpenSearch vs GCP(无原生对标)

AWS · Amazon OpenSearch

定位：托管 OpenSearch 集群，日志分析 / 实时监控 / 点击流

✓ 优点

- 全托管 domain，自动替换故障节点、一键扩缩
- 兼容 OpenSearch 与 legacy Elasticsearch OSS(≤ 7.10)
- 生态含 Dashboards、日志 / 可观测完整

✗ 缺点

- domain 需选实例 / 容量并管理
- 集群规模 / 分片规划仍需经验

GCP · GCP(无原生对标)

定位：无与 OpenSearch 一一对应的第一方托管搜索集群

✓ 优点

- Cloud Logging+Log Analytics(底层 BigQuery) 做检索
- 全文 / 向量搜索用 BQ search 或 Vertex AI Search
- 可在 GCP 部署 Elastic Cloud 替代

✗ 缺点

- 无第一方 Elasticsearch 兼容托管搜索集群
- 需自行组合多个服务或用第三方

选型建议：需 ES/OpenSearch 兼容托管集群→ AWS 明显占优； GCP 侧用 Log Analytics/BQ+Vertex AI Search 或 Elastic Cloud

10. 湖仓 / 表格式

Amazon S3 Tables vs BQ Apache Iceberg managed tables

AWS • Amazon S3 Tables

定位：新 bucket 类型 (table bucket)，原生存 Iceberg+ 自动优化

✓ 优点

- 原生 Iceberg 存储 + 内建自动维护 (compaction)
- 多引擎可查 (Athena/Redshift/Spark)
- 支持 Iceberg V3、跨区复制、智能分层降本

✗ 缺点

- 新 bucket 类型，区域 / 配额限制需核对
- 生态与工具链仍在成熟

GCP • BQ Apache Iceberg managed tables

定位：GCS 上以 Iceberg 开放格式提供与原生 BQ 表一致的托管体验

✓ 优点

- 全托管但数据存客户自有 GCS，兼顾开放 + 治理
- 支持 DML/ 高吞吐流写 / schema 演进 / 时间旅行 / 列级安全
- 自动存储优化 (自适应文件 / 聚簇 / GC)

✗ 缺点

- 强绑 BigQuery 作为管理层
- 跨引擎写入路径受限 (以 BQ 为主写)

选型建议：以 S3 为湖底座 + 开放 Iceberg 多引擎 → S3 Tables；已用 BQ 要托管 + 数据自有 + 开放格式 → BQ Iceberg managed tables

总结：怎么选

一句话决策指南

追求零运维、开箱即分析、内建 AI

→ GCP(BigQuery 为核心)

要灵活组合、生态广度、精细选型

→ AWS(purpose-built 服务群)

需要搜索 / 日志分析 (ES 兼容)

→ AWS OpenSearch(GCP 无原生对标)

需要托管 Kafka

→ AWS MSK(GCP 另有 Managed Kafka)

要批流一体、统一编程模型

→ GCP Dataflow(Beam)

要企业级 BI 语义层

→ GCP Looker(LookML)

要开放中立的托管 Iceberg 湖

→ AWS S3 Tables(存储归存储 , 引擎随便挑)

要存算一体的托管 Iceberg

→ GCP BQ Iceberg managed tables